

Business Case Study: Target

Overview

This case study analyzes Target's e-commerce operations in Brazil using a dataset of 100,000+ orders placed between 2016 and 2018. The data covers multiple aspects of the business, including customers, orders, payments, products, freight, delivery timelines, geolocation, and customer reviews. By examining these datasets together, the goal is to uncover patterns in customer behavior, order trends, payment preferences, logistics performance, and delivery efficiency. The insights aim to help Target optimize its operations, improve customer experience, and identify opportunities for business growth in the Brazilian market.

Problem Statement:

As a data analyst/data scientist at Target, I have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

What does 'good' look like?

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

```
select * from `Target.customers` limit 5;
```

Row	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
1	2201362e68992f654942dc006...	f7d7fc0a59ef4363fdce6e3aa06...	69900	rio branco	AC
2	31dbc13addc753e210692eaca...	5dbba6c01268a8ad43f79157bf...	69900	rio branco	AC
3	dad907e170748a35ef4e92238...	36b1c0516f123351ffa8743041...	69900	rio branco	AC
4	888d2ebe1af2a8c93c75dae5df...	721d1092e1a6460c67e6a0e69...	69900	rio branco	AC
5	8a0108267d9258a0ec9f74381...	7a2dc4682890550ebe3b8befce...	69900	rio branco	AC

```
select * from `Target.geolocation` limit 5;
```

Row	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
1	1037	-23.54562128115268	-46.639292048001678	sao paulo	SP
2	1046	-23.546081127035535	-46.64482029837157	sao paulo	SP
3	1046	-23.546128966414692	-46.642951483611377	sao paulo	SP
4	1041	-23.5443921648681	-46.639499306278438	sao paulo	SP
5	1035	-23.541577961711493	-46.641607223296127	sao paulo	SP

```
select * from `Target.orders` limit 5;
```

Row	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
1	a2e4c44360b4a57bdf22f3a46...	8886130db0ea9e70ba0b03d...	approved	2017-02-06 20:18:17 UTC	2017-02-06 20:30:19 UTC	null	null	2017-03-01 00:00:00 UTC
2	132f1e724165a07f6362532bfb...	b2191912d8ad6eac2e4dc3b6e...	approved	2017-04-25 01:25:34 UTC	2017-04-30 20:32:41 UTC	null	null	2017-05-22 00:00:00 UTC
3	809a282b2bd5dbcab6f2724fc...	622e13439d6b5a0b486c43561...	canceled	2016-09-13 15:24:19 UTC	2016-10-07 13:16:46 UTC	null	null	2016-09-30 00:00:00 UTC
4	e5215415bb6f76fe3b7cb68103...	b6f6cbfc126f1ae6723fe2f9b37...	canceled	2016-10-22 08:25:27 UTC	null	null	null	2016-10-24 00:00:00 UTC
5	71303d7e93b399f5bcd537d12...	b106b360fe2ef8849fbdb056f7...	canceled	2016-10-02 22:07:52 UTC	2016-10-06 15:50:56 UTC	null	null	2016-10-25 00:00:00 UTC

```
select * from `Target.order_items` limit 5;
```

Row	order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
1	3ee6513ae7ea23bdfab5b9ab60...	1	8a3254bee785a526d548a81a9...	96804ea39d96eb908e7c3afdb6...	2018-05-04 03:55:26 UTC	0.85	18.23
2	6e864b3f0ec71031117ad4cf46...	1	8a3254bee785a526d548a81a9...	96804ea39d96eb908e7c3afdb6...	2018-05-02 20:30:34 UTC	0.85	18.23
3	c5b0dd8ef3c0ec4202326668302...	2	8a3254bee785a526d548a81a9...	96804ea39d96eb908e7c3afdb6...	2018-05-07 02:55:22 UTC	0.85	22.3
4	8272b63d03f5f79c55e9e4120a...	2	05b515fdc76e888aada3c6d66c...	2709af9587499e95e803a6498...	2017-07-21 18:25:23 UTC	1.2	7.89
5	8272b63d03f5f79c55e9e4120a...	3	05b515fdc76e888aada3c6d66c...	2709af9587499e95e803a6498...	2017-07-21 18:25:23 UTC	1.2	7.89

```
select * from `Target.order_reviews` limit 5;
```

Row	review_id	order_id	review_score	review_comment_title	review_creation_date	review_answer_timestamp
1	9e628acd082a5eeea001b1ded...	54d744a4410b1edccc36c6d1f1...	1	null	0001-02-17 00:00:00 UTC	0015-03-17 02:47:00 UTC
2	db27636ea82d449a9e0d1bea6...	a9a93c428c6103f2151bb63a1...	1	null	0001-02-17 00:00:00 UTC	0001-02-17 11:12:00 UTC
3	5bec0620e043423a063641e9a...	1cb796218c383fc54a6a45414...	1	null	0001-02-17 00:00:00 UTC	0013-02-17 11:43:00 UTC
4	3c9d8bbd2a92badcc2356f4c8e...	b3feb3846bb0a8d68cd328138...	1	null	0001-02-18 00:00:00 UTC	0002-02-18 10:36:00 UTC
5	647a5c7d693aedb5ce12f3c5e5...	745e2506fb647deca4669e1c88...	1	null	0001-02-18 00:00:00 UTC	0001-02-18 21:10:00 UTC

```
select * from `Target.payments` limit 5;
```

Row	order_id	payment_sequent...	payment_type	payment_installm...	payment_value
1	744bade1fcf9ff3f31d860ace07...	2	credit_card	0	58.69
2	1a57108394169c0b47d8f876a...	2	credit_card	0	129.94
3	8bcbe01d44d147f901cd31926...	4	voucher	1	0.0
4	fa65dad1b0e818e3ccc5cb0e39...	14	voucher	1	0.0
5	6ccb433e00daae1283ccc9561...	4	voucher	1	0.0

```
select * from `Target.orders` limit 5;
```

Row	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
1	a2e4c44360b4a57bdf22f3a46...	8886130db0ea9e70ba0b03d...	approved	2017-02-06 20:18:17 UTC	2017-02-06 20:30:19 UTC	null	null	2017-03-01 00:00:00 UTC
2	132f1e724165a07f6362532bfb...	b2191912d8ad6eac2e4dc3b6e...	approved	2017-04-25 01:25:34 UTC	2017-04-30 20:32:41 UTC	null	null	2017-05-22 00:00:00 UTC
3	809a282b2bd5dbcab6f2724fc...	622e13439d6b5a0b486c43561...	canceled	2016-09-13 15:24:19 UTC	2016-10-07 13:16:46 UTC	null	null	2016-09-30 00:00:00 UTC
4	e5215415bb6f76fe3b7cb68103...	b6f6cbfc126f1ae6723fe2f9b37...	canceled	2016-10-22 08:25:27 UTC	null	null	null	2016-10-24 00:00:00 UTC
5	71303d7e93b399f5bcd537d12...	b106b360fe2ef8849fbdb056f7...	canceled	2016-10-02 22:07:52 UTC	2016-10-06 15:50:56 UTC	null	null	2016-10-25 00:00:00 UTC

```
select * from `Target.products` limit 5;
```

Row	product_id	product_category	product_name_length	product_description_length	product_photos...	product_weight_g	product_length_cm	product_height_cm	product_width_cm
1	5eb564652db742ff8f28759cd8...	null	null	null	null	null	null	null	null
2	09ff539a621711667c43eba6a3...	babies	60	865	3	null	null	null	null
3	a69f15dfb803d485e8933e80b9...	Watches present	53	506	6	150	11	16	6
4	2f763ba79d9cd987b2034aac7...	electronics	45	1198	2	595	8	6	6
5	106392145fca363410d287a81...	bed table bath	58	309	1	2083	12	2	7

```
select * from `Target.sellers` limit 5;
```

Row	seller_id	seller_zip_code_prefix	seller_city	seller_state
1	c13ef0cfbe42f190780f621ce81...	1207	sao paulo sp	SP
2	5444b12c82f21c923f2639ebc7...	2051	sao paulo	SP
3	1cbd32d00d01bb8087a5eb088...	3363	sp / sp	SP
4	71593c7413973a1e160057b80...	3407	sao paulo / sao paulo	SP
5	6f1a1263039c76e68f40a8e536...	3581	sao paulo	SP

1. Data type of all columns in the "customers" table.

customers

☆

Query

Open in ▾

Share ▾

Copy

S

Schema

Details

Preview

Table Explorer

Preview

Insights

Linea

≡

Filter

Enter property name or value

☐	Field name	Type	Mode	Description
☐	customer_id	STRING	NULLABLE	-
☐	customer_unique_id	STRING	NULLABLE	-
☐	customer_zip_code_prefix	INTEGER	NULLABLE	-
☐	customer_city	STRING	NULLABLE	-
☐	customer_state	STRING	NULLABLE	-

Solution:

```
select
    column_name,
    data_type
from `Target.INFORMATION_SCHEMA.COLUMNS`
where table_name = 'customers';
```

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

2. Get the time range between which the orders were placed.

Solution:

```
select
    min(order_purchase_timestamp) as earliest_order,
    max(order_purchase_timestamp) as latest_order
from `Target.orders`;
```

Row	earliest_order	latest_order
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

3. Count the Cities & States of customers who ordered during the given period.

Solution 1:

```
select
    count(distinct c.customer_city) as num_cities,
    count(distinct c.customer_state) as num_states
from `Target.customers` c join `Target.orders` o
    on c.customer_id = o.customer_id
where o.order_purchase_timestamp between
    (select min(o.order_purchase_timestamp) from `Target.orders`) and
    (select max(o.order_purchase_timestamp) from `Target.orders`);
```

Row	num_cities	num_states
1	4119	27

Solution 2:

```
select
    count(distinct c.customer_city) as num_cities,
    count(distinct c.customer_state) as num_states
from `Target.customers` c
join `Target.orders` o
    on c.customer_id = o.customer_id;
```

Row	num_cities	num_states
1	4119	27

2. In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

Solution :

```
select format_date('%Y', order_purchase_timestamp) as year,
```

```

        count(*) as yearly_order_count
from `Target.orders`
group by format_date('%Y', order_purchase_timestamp)
order by year;

```

Row	year	yearly_order_count
1	2016	329
2	2017	45101
3	2018	54011

```

with yearly_purchase_stats as (
    select
        format_date('%Y', order_purchase_timestamp) as year,
        customer_city,
        count(*) as yearly_order_count,
    from `Target.customers` c join `Target.orders` o
        on c.customer_id = o.customer_id
    group by
        format_date('%Y', order_purchase_timestamp),
        customer_city
),
ranked_yearly_purchase_stats as (
    select
        year,
        customer_city,
        yearly_order_count,
        dense_rank() over(partition by year order by yearly_order_count
desc) as order_count_rank
    from yearly_purchase_stats
)

select
    year,
    customer_city,
    yearly_order_count
from ranked_yearly_purchase_stats
where order_count_rank between 1 and 3
order by year, order_count_rank;

```

Job information		Results	Visualization	JSON	Execution details	Execution graph
Row	year ▼	customer_city ▼	yearly_order_count ▼			
1	2016	rio de janeiro	38			
2	2016	sao paulo	36			
3	2016	belo horizonte	12			
4	2017	sao paulo	6381			
5	2017	rio de janeiro	3341			
6	2017	belo horizonte	1206			
7	2018	sao paulo	9123			
8	2018	rio de janeiro	3503			
9	2018	belo horizonte	1555			

Insights:

- There is a strong upward trend in the number of orders year over year.
- The growth is especially sharp between 2016 to 2017, which could indicate either an expansion in user base, better promotions, or more product availability.
- These three cities — São Paulo, Rio de Janeiro, and Belo Horizonte consistently made the maximum purchases in the years 2016, 2017, 2018

Recommendations

- Expand Inventory & Logistics in these cities
- Improve delivery speed in these regions

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Solution :

```
select
    format_date('%B', order_purchase_timestamp) as month,
    count(order_id) as total_orders
from `Target.orders`
group by format_date('%B', order_purchase_timestamp)
```

```
order by parse_date('%B', month);
```

Job information		Results	Visualization	JSON	Execution details	Ex
Row	month ▼	total_orders ▼				
1	January	8069				
2	February	8508				
3	March	9893				
4	April	9343				
5	May	10573				
6	June	9412				
7	July	10318				
8	August	10843				
9	September	4305				
10	October	4959				
11	November	7544				
12	December	5674				

Insights

- Orders stay high and steady between Feb and Aug.
- Then a sharp drop happens around Sep - Oct.

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

- 0-6 hrs : Dawn
- 7-12 hrs : Mornings
- 13-18 hrs : Afternoon
- 19-23 hrs : Night

Solution :

```
select
    case
        when extract(hour from order_purchase_timestamp) between 0 and 6
        then 'Dawn'
        when extract(hour from order_purchase_timestamp) between 7 and 12
        then 'Mornings'
        when extract(hour from order_purchase_timestamp) between 13 and
        18 then 'Afternoon'
        when extract(hour from order_purchase_timestamp) between 19 and
        23 then 'Night'
        end as time_of_the_day,
    count(*) as no_of_orders_placed
from `Target.orders`
```

```

group by (
    case
        when extract(hour from order_purchase_timestamp) between 0 and 6
then 'Dawn'
        when extract(hour from order_purchase_timestamp) between 7 and 12
then 'Mornings'
        when extract(hour from order_purchase_timestamp) between 13 and
18 then 'Afternoon'
        when extract(hour from order_purchase_timestamp) between 19 and
23 then 'Night'
    end
);

```

Row	time_of_the_day ▼	no_of_orders_pla...
1	Night	28331
2	Dawn	5242
3	Afternoon	38135
4	Mornings	27733

Insights:

- Customers are most active between 13:00–18:00, which suggests higher engagement after lunch and before end-of-day routines.
- 0:00–06:00 has minimal activity.

Recommendation:

- Schedule push notifications and promotions in the afternoon, so more of them will buy something.

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month on month no. of orders placed in each state.

Solution 2:

```

select
    c.customer_state as state,
    extract(year from o.order_purchase_timestamp) as year,
    format_date('%B', o.order_purchase_timestamp) as month,
    count(*) as no_of_orders
from `Target.orders` o join `Target.customers` c
    on o.customer_id = c.customer_id
group by

```



```

        c.customer_state,
        extract(year from o.order_purchase_timestamp),
        format_date('%B', o.order_purchase_timestamp)
order by state, year, month;

```

Row	state ▼	year ▼	month ▼	no_of_orders ▼	
1	AC	2017	April	5	
2	AC	2017	August	4	
3	AC	2017	December	5	
4	AC	2017	February	3	
5	AC	2017	January	2	
6	AC	2017	July	5	
7	AC	2017	June	4	
8	AC	2017	March	2	
9	AC	2017	May	8	

Insights

- States like SP, RJ, MG consistently receive the highest number of monthly orders.

Recommendation:

- Expand logistics & inventory in these states to handle high order volumes efficiently.
- Targeted marketing & promotions in these regions could further boost sales, since customer engagement is already high.

2. How are the customers distributed across all the states?

Solution:

```

select
    customer_state as state,
    count(*) as customer_count
from `Target.customers` c
group by customer_state
order by customer_count desc;

```

Job information		Results	Visualization	JSON	Exe
Row	state ▼	customer_count ▼			
1	SP	41746			
2	RJ	12852			
3	MG	11635			
4	RS	5466			
5	PR	5045			
6	SC	3637			

Insights:

- Most customers are concentrated in SP, RJ, and MG

Recommendation:

- Since most customers live in SP, RJ, and MG, opening more stores or improving delivery in these states would bring the best results

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table to get the cost of orders.

Solution :

```
with payments_per_order as (
  select
    order_id,
    sum(payment_value) as total_amount
  from `Target.payments`
  group by order_id
),
orders_filtered as (
  select
    order_id,
    extract(year from order_purchase_timestamp) as year,
    extract(month from order_purchase_timestamp) as month,
    order_status
  from `Target.orders`
  where extract(year from order_purchase_timestamp) between 2017 and 2018
```

```

        and extract(month from order_purchase_timestamp) between 1 and 8
        and order_status not in ('canceled')
    ),
    yearly_payment_stats as (
        select
            o.year,
            sum(total_amount) as yearly_total_amount
        from orders_filtered o join payments_per_order p
            on o.order_id = p.order_id
        group by o.year
    ),
    compare_totals as (
        select
            *,
            lag(yearly_total_amount) over(order by year) as prev_year_total
        from yearly_payment_stats
    )

select year,
       compare_totals.yearly_total_amount,
       round((yearly_total_amount - prev_year_total) * 100 /
prev_year_total, 2) as percentage_increase
from compare_totals

```

Row	year	yearly_total_amount	percentage_increase
1	2017	3627841.5599999209	null
2	2018	8624464.940000017	137.73

Insight:

- People spent a lot more in 2018 — about 138% more than 2017.

2. Calculate the Total & Average value of order price for each state.

```

with payments_per_order as (
    select
        order_id,
        sum(payment_value) as order_total
    from `Target.payments`
    group by order_id
)

select

```

```

        c.customer_state,
        round(sum(p.order_total), 2) as total_order_price,
        round(avg(p.order_total), 2) as avg_order_price
from `Target.orders` o join payments_per_order p
    on o.order_id = p.order_id
join `Target.customers` c
    on o.customer_id = c.customer_id
where o.order_status not in ('canceled')
group by c.customer_state
order by total_order_price desc;

```

Row	customer_state	total_order_price	avg_order_price	
1	SP	5942397.11	143.47	
2	RJ	2126444.23	166.57	
3	MG	1856375.81	160.43	
4	RS	881680.6	162.04	
5	PR	802319.18	159.73	
6	SC	613707.46	169.63	
7	BA	611796.01	181.87	
8	DF	352718.04	165.36	
9	GO	342124.8	170.47	
10	ES	324039.0	160.1	

Results per pag

Insight:

- SP has the highest overall order value, which makes sense because it also has the biggest customer base.

3. Calculate the Total & Average value of order freight for each state.

Solution:

```

with freights_per_order as (
    select
        order_id,
        sum(freight_value) as order_freight_value
    from `Target.order_items`
    group by order_id
)

select
    c.customer_state,

```

```

        round(sum(f.order_freight_value), 2) as total_freight_price,
        round(avg(f.order_freight_value), 2) as avg_freight_price
from `Target.orders` o join freights_per_order f
    on o.order_id = f.order_id
join `Target.customers` c
    on o.customer_id = c.customer_id
where o.order_status not in ('canceled')
group by c.customer_state
order by total_freight_price desc;

```

Row	customer_state	total_freight_price	avg_freight_price
1	SP	714329.91	17.37
2	RJ	304057.82	23.95
3	MG	269599.43	23.45
4	RS	134751.66	24.88
5	PR	117313.55	23.55
6	BA	99799.83	29.84
7	SC	89445.42	24.85
8	PE	59278.4	36.08
9	GO	52673.91	26.36

Insight:

- SP, RJ, MG spend the most overall on shipping — since they have the largest number of orders.

5. Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- $\text{time_to_deliver} = \text{order_delivered_customer_date} - \text{order_purchase_timestamp}$
- $\text{diff_estimated_delivery} = \text{order_delivered_customer_date} - \text{order_estimated_delivery_date}$

Solution :

```
select
    order_id,
    timestamp_diff(order_delivered_customer_date,
order_purchase_timestamp, day) as time_to_deliver,
    timestamp_diff(order_delivered_customer_date,
order_estimated_delivery_date, day) as diff_estimated_delivery
from `Target.orders`
where order_status = 'delivered';
```

Row	order_id	time_to_deliver	diff_estimated_delivery
1	bfb0f9bdef84302105ad712db...	54	36
2	98974b076b01553d49ee64679...	43	-6
3	c4b41c36dd589e901f6879f25a...	36	-14
4	d2292ff2201e74c5db154d1b7a...	29	-20
5	95e01270fcbae986342340010...	30	-19
6	ed8c7b1b3eb256c70ce0c7423...	44	-5
7	5cc475c7c03290048eb2e742c...	68	18
8	6b3ee7697a02619a0ace2b3f0a...	47	-2
9	3b2ca3293a7ce539ea2379d70...	43	-7

Insights:

- delay_vs_estimate_days
 - negative number -> delivered before estimated date
 - zero -> delivered exactly on the estimated date
 - positive -> delivered after estimated date (late)
- Many orders were delivered before the delivery estimate, while only a few were late. This means the delivery system generally performs better than expected.

2. Find out the top 5 states with the highest & lowest average freight value.

Solution1:

```
with statewise_freight as (
    select c.customer_state as state,
           avg(oi.freight_value) as avg_freight_value
    from `Target.orders` as o
```

```

join `Target.order_items` as oi
  on o.order_id = oi.order_id
join `Target.customers` as c
  on o.customer_id = c.customer_id
group by c.customer_state
),
ordered_values as (
  select
    state,
    statewise_freight.avg_freight_value,
    row_number() over(order by statewise_freight.avg_freight_value
desc) as rank_desc,
    row_number() over(order by statewise_freight.avg_freight_value
asc) as rank_asc
  from statewise_freight
)
select
  case
    when rank_desc <= 5 then 'TOP'
    when rank_asc <= 5 then 'BOTTOM'
  end as category,
  state,
  avg_freight_value
from ordered_values
where rank_desc <= 5
  or rank_asc <= 5
order by avg_freight_value desc;

```

Row	category	state	avg_freight_value
1	TOP	RR	42.98442307692...
2	TOP	PB	42.72380398671...
3	TOP	RO	41.06971223021...
4	TOP	AC	40.07336956521...
5	TOP	PI	39.14797047970...
6	BOTTOM	DF	21.04135494596...
7	BOTTOM	RJ	20.96092393168...
8	BOTTOM	MG	20.63016680630...
9	BOTTOM	PR	20.53165156794...
10	BOTTOM	SP	15.14727539041...

Solution 2:

```

with statewise_freight as (
  select c.customer_state as state,
    avg(oi.freight_value) as avg_freight_value

```

```

from `Target.orders` as o
join `Target.order_items` as oi
  on o.order_id = oi.order_id
join `Target.customers` as c
  on o.customer_id = c.customer_id
group by c.customer_state
),
tops as (
  select 'TOP' as category, *
  from statewise_freight
  order by avg_freight_value desc
  limit 5
),
bottoms as (
  select 'BOTTOM' as category, *
  from statewise_freight
  order by avg_freight_value asc
  limit 5
)
select *
from (
  select * from tops
  union all
  select * from bottoms
)
order by avg_freight_value desc;

```

Row	category	state	avg_freight_value
1	TOP	RR	42.98442307692...
2	TOP	PB	42.72380398671...
3	TOP	RO	41.06971223021...
4	TOP	AC	40.07336956521...
5	TOP	PI	39.14797047970...
6	BOTTOM	DF	21.04135494596...
7	BOTTOM	RJ	20.96092393168...
8	BOTTOM	MG	20.63016680630...
9	BOTTOM	PR	20.53165156794...
10	BOTTOM	SP	15.14727539041...

3. Find out the top 5 states with the highest & lowest average delivery time.

Solution:

```
with delivery_stats as (  
    select  
        c.customer_state as state,  
        timestamp_diff(o.order_delivered_customer_date,  
o.order_purchase_timestamp, day) as time_to_deliver  
    from `Target.orders` as o  
    join `Target.customers` as c  
        on o.customer_id = c.customer_id  
)  
,  
average_delivery_stats as (  
    select  
        state,  
        avg(time_to_deliver) as avg_delivery_time  
    from delivery_stats  
    group by state  
)  
,  
ordered_delivery_time as (  
    select  
        state,  
        avg_delivery_time,  
        row_number() over(order by avg_delivery_time desc) as  
rank_desc,  
        row_number() over(order by avg_delivery_time asc) as rank_asc  
    from average_delivery_stats  
)  
select  
    state,  
    round(avg_delivery_time, 2) as avg_delivery_time,  
    case  
        when rank_desc <= 5 then 'TOP'  
        when rank_asc <= 5 then 'BOTTOM'  
    end as category  
from ordered_delivery_time  
where rank_desc <= 5  
    or rank_asc <= 5  
order by avg_delivery_time desc;
```

Row	state	avg_delivery_time	category
1	RR	28.98	TOP
2	AP	26.73	TOP
3	AM	25.99	TOP
4	AL	24.04	TOP
5	PA	23.32	TOP
6	SC	14.48	BOTTOM
7	DF	12.51	BOTTOM
8	MG	11.54	BOTTOM
9	PR	11.53	BOTTOM
10	SP	8.3	BOTTOM

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Solution:

```
select
    c.customer_state as state,
    round(
        avg(timestamp_diff(o.order_delivered_customer_date,
o.order_estimated_delivery_date, day)),
        2) as avg_diff_estimated_delivery
from `Target.orders` o
join `Target.customers` c
    on o.customer_id = c.customer_id
where o.order_status = 'delivered'
group by c.customer_state
order by avg_diff_estimated_delivery asc
limit 5;
```

Row	state	avg_diff_estimated_delivery
1	AC	-19.76
2	RO	-19.13
3	AP	-18.73
4	AM	-18.61
5	RR	-16.41

Insights:

- These states consistently receive orders 16–20 days earlier than expected, indicating overestimated delivery times and fast logistics performance.

6. Analysis based on the payments:

1. Find the month on month no. of orders placed using different payment types.

Solution:

```
select
    p.payment_type,
    sum(case when extract(month from o.order_purchase_timestamp) = 1 then
1 else 0 end) as jan,
    sum(case when extract(month from o.order_purchase_timestamp) = 2 then
1 else 0 end) as feb,
    sum(case when extract(month from o.order_purchase_timestamp) = 3 then
1 else 0 end) as mar,
    sum(case when extract(month from o.order_purchase_timestamp) = 4 then
1 else 0 end) as apr,
    sum(case when extract(month from o.order_purchase_timestamp) = 5 then
1 else 0 end) as may,
    sum(case when extract(month from o.order_purchase_timestamp) = 6 then
1 else 0 end) as jun,
    sum(case when extract(month from o.order_purchase_timestamp) = 7 then
1 else 0 end) as jul,
    sum(case when extract(month from o.order_purchase_timestamp) = 8 then
1 else 0 end) as aug,
    sum(case when extract(month from o.order_purchase_timestamp) = 9 then
1 else 0 end) as sep,
    sum(case when extract(month from o.order_purchase_timestamp) = 10 then
1 else 0 end) as oct,
    sum(case when extract(month from o.order_purchase_timestamp) = 11 then
1 else 0 end) as nov,
    sum(case when extract(month from o.order_purchase_timestamp) = 12 then
1 else 0 end) as dec
from `Target.orders` o join `Target.payments` p
    on o.order_id = p.order_id
group by
    p.payment_type
```

Row	payment_type ▾	jan ▾	feb ▾	mar ▾	apr ▾	may ▾	jun ▾	jul ▾	aug ▾	sep ▾	oct ▾	nov ▾	dec ▾
1	credit_card	6103	6609	7707	7301	8350	7276	7841	8269	3286	3778	5897	4378
2	voucher	477	424	591	572	613	563	645	589	302	318	387	294
3	UPI	1715	1723	1942	1783	2035	1807	2074	2077	903	1056	1509	1160
4	debit_card	118	82	109	124	81	209	264	311	43	54	70	64
5	not_defined	0	0	0	0	0	0	0	2	1	0	0	0

Insights:

- Credit cards are the most used payment method every month.
- UPI is the second most preferred option and is used consistently throughout the year.
- Vouchers are used only by a small set of customers, mostly during mid-year months.
- Debit cards are used very rarely compared to credit cards and UPI.
- “Not defined” is almost zero and likely due to missing data.
- Overall order counts seem high from January to August and then drop around September to December.

Recommendation:

- Give extra reward points to frequent credit-card and UPI users so they stay engaged and keep choosing these payment methods.

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

Solution:

```
select
    payment_installments,
    count(order_id) as order_count
from `Target.payments` p
group by payment_installments
order by payment_installments;
```

Row	payment_installments ▼	order_count ▼	
1	0	2	
2	1	52546	
3	2	12413	
4	3	10461	
5	4	7098	
6	5	5239	
7	6	3920	
8	7	1626	

Insights:

- Customers largely prefer paying in full or using small installment plans. Large installment plans are not commonly used.

Key Findings & Actionable Recommendations

Insights

- Strong upward trend in order volume from 2016–2018, especially 2016–2017.
- São Paulo (SP), Rio de Janeiro (RJ), and Minas Gerais (MG) dominate across metrics — they have the highest customer concentration, the most orders, and the largest total order + freight values.
- Orders are high from Feb to Aug, with a drop in Sep–Oct.
- Most orders are placed in the afternoon (13:00–18:00).
- Credit cards are the most used payment method, followed by UPI.
- Most customers prefer full payment or small installments.
- Many orders are delivered before the estimated delivery date, with some states 16–20 days earlier.

Recommendations

- Expand inventory, logistics, and marketing in SP, RJ, and MG.
- Schedule push notifications and promotions in the afternoon.
- Prepare for peak months (Feb–Aug) with sufficient stock and delivery capacity.
- Offer rewards or cashback for frequent credit card and UPI users.
- Keep installment plans simple, avoid long-term installments.
- Optimize freight and shipping in high-order states to control costs.